

Project Report: Toxic Comment Classification

Program: IBM SkillsBuild Data Analytics with AI Academic Internship Program

Organized by: BharatCares in Association with AICTE

Student Name: Shimpi

Project Title: Toxic Comment Classification using Natural Language Processing and Machine Learning

1. Introduction & Executive Summary

With the rapid growth of online communication, platforms struggle to facilitate constructive and respectful conversations due to cyberbullying, harassment, and toxic language. Manual moderation is slow and inefficient, making automated content moderation crucial.

This project implements a multi-headed machine learning and Natural Language Processing (NLP) pipeline to detect and classify toxic comments from Wikipedia's talk page edits. The system categorizes text into multiple dimensions of toxicity: **toxic**, **severe_toxic**, **obscene**, **threat**, **insult**, and **identity_hate**. Using TF-IDF vectorization combined with robust classifiers like **Linear SVC** and **Logistic Regression**, the model achieves high F1-scores and low Hamming loss, offering a reliable, scalable solution for automated text moderation.

2. Problem Statement & Dataset Overview

- **Objective:** Build a multi-label classification model capable of distinguishing between clean comments and various forms of toxic behavior.
 - **Dataset Source:** Wikipedia talk page edits collected via Kaggle ([train.csv](#), [test.csv](#), and [test_labels.csv](#)).
 - **Dataset Dimensions:**
 - Training Dataset: 159,571 observations
 - Testing Dataset: 153,164 observations
 - **Target Labels:** [toxic](#), [severe_toxic](#), [obscene](#), [threat](#), [insult](#), [identity_hate](#).
 - **Key Challenge:** Severe class imbalance. The vast majority of comments are clean (non-toxic), while severe categories like [threat](#) and [identity_hate](#) represent a very small fraction of the training observations.
-

3. Data Preprocessing & Exploratory Data Analysis (EDA)

To prepare raw text data for machine learning models, a thorough preprocessing pipeline was implemented:

1. **Text Tokenization & Cleaning:** A custom `tokenize()` function was designed to strip punctuations, remove special characters, filter out short comments (length < 3), and apply stemming/lemmatization using NLTK (`WordNetLemmatizer`).
 2. **Feature Extraction:** Evaluated and compared `CountVectorizer` and `TfidfVectorizer`. **TF-IDF Vectorization** was selected as it delivered superior performance by weighting terms based on inverse document frequency.
 3. **Exploratory Insights:**
 - Analysis of comment length frequency showed that most comments are relatively short, with very few exceeding 1,000 words.
 - Class distribution plots confirmed extreme class imbalance, requiring careful metric selection (F1-score, Recall, Hamming Loss) and robust threshold tuning to handle rare classes effectively.
-

4. Modeling & Machine Learning Pipeline

The modeling phase followed an iterative approach, progressing from baseline models to hyperparameter-tuned pipelines and ensembling.

A. Baseline & Basic Model Comparison

- **Models Tested:** Multinomial Naive Bayes, Logistic Regression, and Linear SVC.
- **Evaluation Metrics:** F1-Score, Recall, and Hamming Loss.
- **Findings:** Multinomial Naive Bayes underperformed compared to the linear models. **Linear SVC** and **Logistic Regression** demonstrated strong initial performance, with Linear SVC achieving the fastest training time and highest baseline F1 score.

B. Pipeline with Manual Hyperparameter Tuning & Grid Search

- Incorporating class balancing strategies and optimized hyperparameter tuning significantly boosted performance.
- Logistic Regression reached an average F1-score of **0.9566**, while Linear SVC achieved an optimal average F1-score of **0.9585**.

C. Ensembling Techniques

- Explored tree-boosting models (AdaBoost, Gradient Boosting, XGBoost) and constructed a **Voting Classifier** combining boosted models with linear estimators.
 - The ensembled pipeline achieved an exceptional F1-score of **0.9735** and a low Hamming Loss of **0.0246**.
-

5. Results & Model Selection

Although ensembling yielded high accuracy, **Linear SVC** was selected as the optimal production model for the following reasons:

- **Training Speed:** Trains substantially faster than complex tree ensembles or deep neural networks.
- **Interpretability:** Easier to inspect coefficients, feature weights, and understand internal decision boundaries.
- **Performance:** Delivered competitive F1-scores with minimal complexity, making it ideal for real-time deployment.

Feature Importance (Top & Bottom Features)

Analysis of model coefficients revealed distinct linguistic patterns:

- **Negative/Neutral Indicators:** Words such as *article*, *talk*, *thanks*, and *source* strongly correlated with clean, constructive comments.
 - **Toxic Indicators:** Profanities, slurs, and aggressive terms dominated the top positive feature weights for toxic and hate-speech categories.
-

6. Conclusion & Future Scope

This project successfully demonstrates the implementation of an end-to-end NLP classification pipeline for online content moderation. By pairing TF-IDF feature extraction with Linear SVC and rigorous preprocessing, the system effectively mitigates class imbalance and accurately flags multiple forms of online toxicity.